



CS-343: Web Technologies

Dr. Naima Iltaf

Tenured Professor

Naima.iltaf@seecs.edu.pk

Lecture-7

Spring-2026

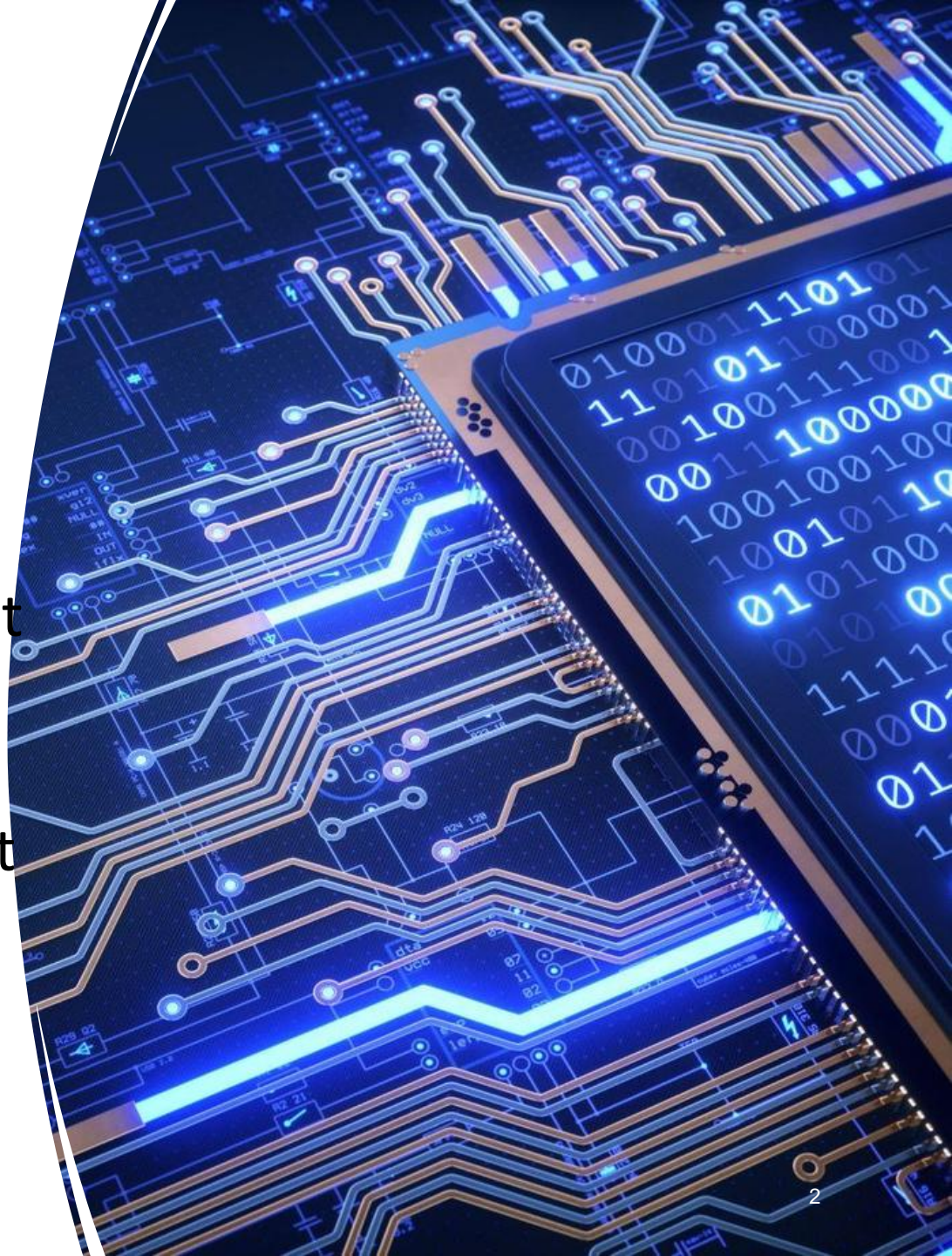
**School of Electrical Engineering and
Computer Science**

**National University of Science and
Technology (NUST)**

Agenda

JavaScript

- Finding HTML Element
- Modifying DOM
- DOM Manipulation
- Creating New Element



Finding HTML Elements

Query Selector

```
document.querySelector("myId / myClass / tag")
```

//returns first element

```
document.querySelectorAll("myId / myClass / tag")
```

//returns a NodeList

JavaScript function to get the values of First and Last name of form.

```
<body>
<form id="form1" onsubmit= "return getFormvalue()">
First name: <input type="text" name="fname" value="Ahmad"><br>
Last name: <input type="text" name="lname" value="Ali"><br>
<input type="submit" value="Submit">
</form> </body> </html>
```

```
function getFormvalue() {
var firstname = (document.getElementsByName("fname"))[0].value;
var secondname = (document.getElementsByName("lname"))[0].value;
alert(firstname+ " " + secondname);
}
```

Document Object: Use getElementById()

```
<html>
  <head>
    <script type="text/javascript">
      function getElement() {
        var x=document.getElementById("myHeader")
        alert("I am a " + x.tagName + " element")    }
      </script> </head>
  <body>
    <h1 id="myHeader" onclick="getElement()">Click to see
    what element I am!</h1>
  </body>
</html>
```

Modifying DOM

Properties

- **tagName** : returns tag for element nodes
- **innerText** : returns the text content of the element and all its children
- **innerHTML** : returns the plain text or HTML contents in the element
- **textContent** : returns textual content even for hidden elements




```

<body>
  <div>
    <h3>Fruits</h3>
    <ul>
      <li>mango</li>
      <li>orange</li>
      <li>litchi</li>
    </ul>
  </div>
  <script src="script.js"></script>

```

Fruits

- mango
- orange
- litchi

```

> div script.js:2
div.innerText
'Fruits\nmango\norange\nlitchi'
div.innerHTML
'\n    <h3>Fruits</h3>\n    <ul>\n      <li>m
ango</li>\n      <li>orange</li>\n      <li>lit
chi</li>\n    </ul>\n'

```

Fruits

- mango
- orange
- litchi

```
8 </head>
9 <body>
10   <div>
11     <h3>Fruits</h3>
12     <ul>
13       <li>mango</li>
14       <li>orange</li>
15       <li>litchi</li>
16     </ul>
17   </div>
18   <script src="script.js"></script>
19 </body>
```

► div

script.js:2

> div.innerText

< 'Fruits\nmango\norange\nlitchi'

> div.innerHTML

< '\n <h3>Fruits</h3>\n \n mango\n orange\n litchi\n \n '

heading

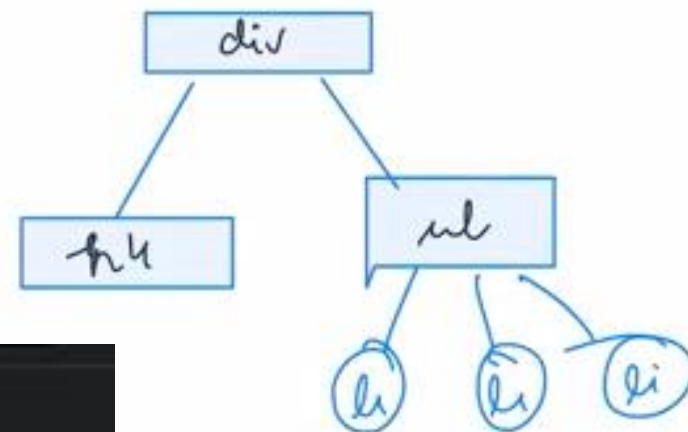
<h1 style="visibility: hidden">old Heading</h1>

heading.innerText

''

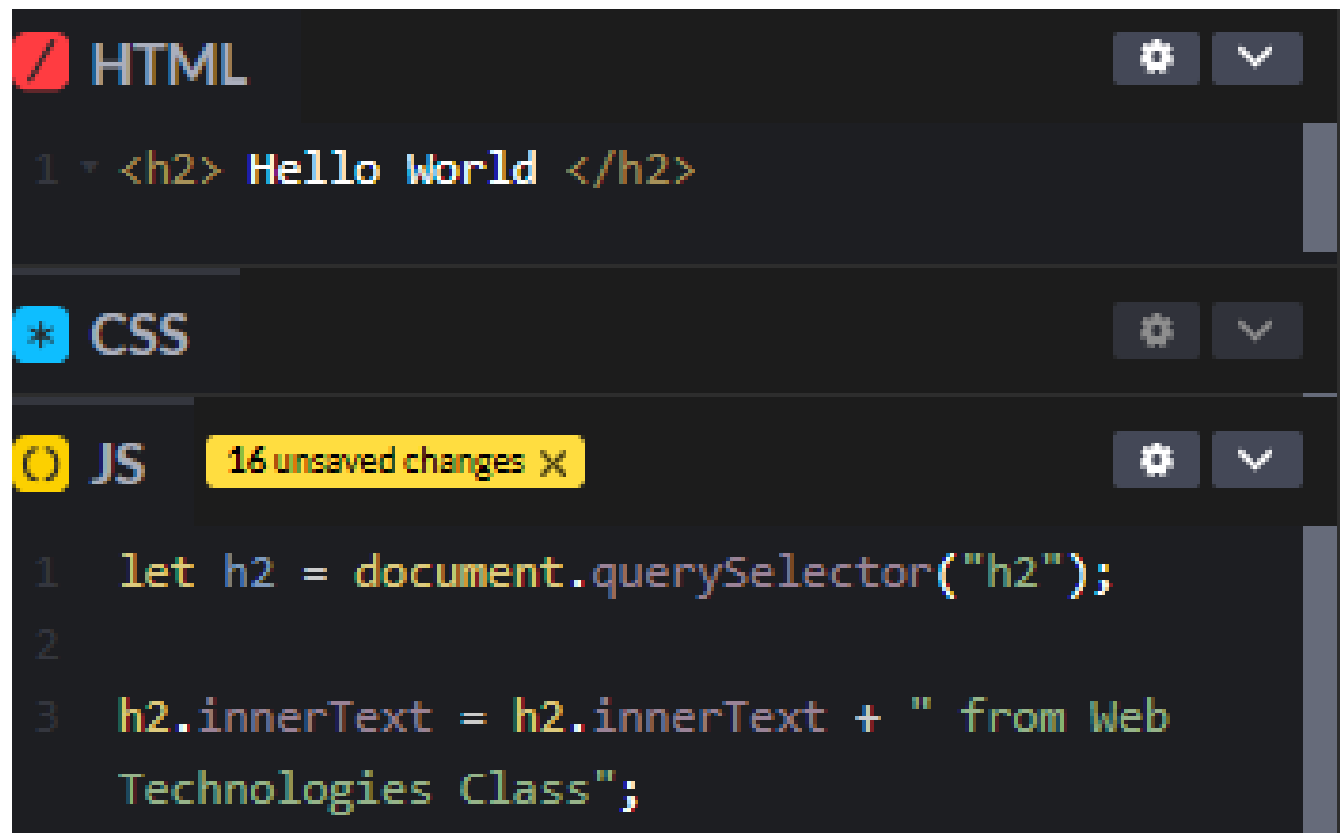
heading.textContent

'old Heading'



Practice Question

- Create a H2 heading element with text – “Hello JavaScript”. Append “from Web Technologies Class” to this text using JS.



The screenshot shows a web development IDE with three panels: HTML, CSS, and JS. The HTML panel contains a single line of code: `<h2> Hello World </h2>`. The CSS panel is empty. The JS panel contains two lines of code: `let h2 = document.querySelector("h2");` and `h2.innerText = h2.innerText + " from Web Technologies Class";`. The JS panel also has a yellow notification bar that says "16 unsaved changes X".

```
HTML
1 <h2> Hello World </h2>

CSS

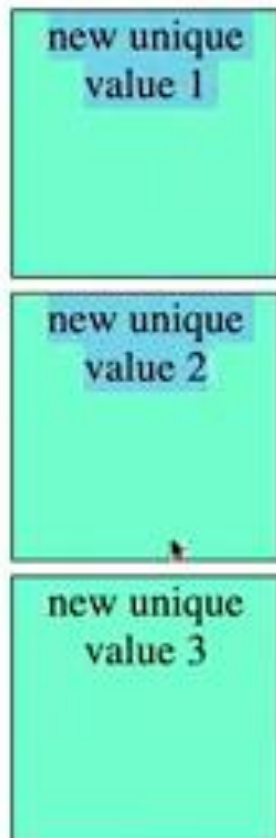
JS 16 unsaved changes X
1 let h2 = document.querySelector("h2");
2
3 h2.innerText = h2.innerText + " from Web
  Technologies Class";
```

Practice Question

- Create a H2 heading element with text – “Hello JavaScript”. Append “from Web Technologies Class” to this text using JS.

Practice Question

- Create 3 divs with common class name – “box”. Access them & add some unique text to each of them.



```
let divs = document.querySelectorAll(".box");  
divs[0].innerText = "new unique value 1";  
divs[1].innerText = "new unique value 2";  
divs[2].innerText = "new unique value 3";
```

```
let idx = 1;  
for (div of divs) {  
  div.innerText = `new unique value ${idx}`;  
  idx++;  
}
```

Example

```
<html>
```

```
<body>
```

```
<p id="intro">Hello World!</p>
```

```
<p>This example demonstrates the <b>getElementById</b> method!</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
    var myElement = document.getElementById("intro");
```

```
    document.getElementById("demo").innerHTML =
```

```
    "The text from the intro paragraph is " + myElement.innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

Hello World!

This example demonstrates the

getElementById method!

The text from the intro paragraph is

Hello World!

JavaScript function to add rows to a table

```
<body>

<table id="sampleTable" border="1">
<tr>
<td>Row1 cell1</td>
<td>Row1 cell2</td>
</tr>

<tr>
<td>Row2 cell1</td>
<td>Row2 cell2</td>
</tr>
</table>

<br><br>

<input type="button" onclick="insert_Row()" value="Insert row">
```

- When the user clicks Insert row,
- JavaScript should: Find the table
- Add a new row at the end
- Add two cells
- Put text inside them ("New Cell1", "New Cell2")

JavaScript function to add rows to a table

When the user clicks Insert row, JavaScript should

- Find the table
- Add a new row at the end
- Add two cells
- Put text inside them ("New Cell1", "New Cell2")

```
<script>
function insert_Row() {

    var x = document.getElementById("sampleTable").insertRow(-1);

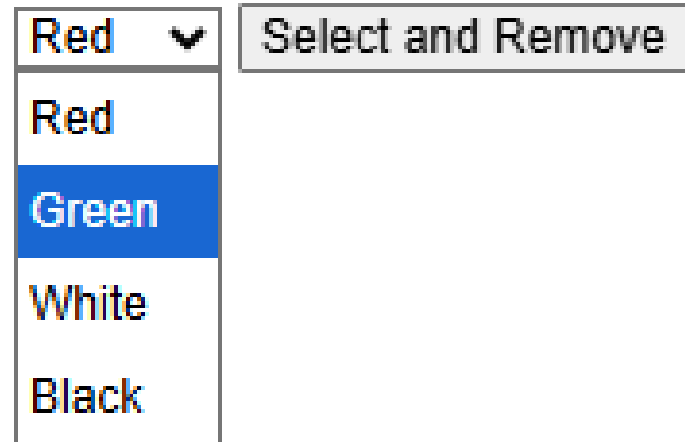
    var y = x.insertCell(0);
    var z = x.insertCell(1);

    y.innerHTML = "New Cell1";
    z.innerHTML = "New Cell2";
}
</script>
```

JavaScript: Remove items from a dropdown list

```
<html>
<head>
  <style type="text/css">
    body {margin: 30px;}
  </style>
  <title>Remove items from a dropdown list
  </title>
</head>
<body>
  <form>
    <select id="colorSelect">
      <option>Red</option>
      <option>Green</option>
      <option>White</option>
      <option>Black</option>
    </select>
    <input type="button" onclick="removecolor ( )"
      value="Select and Remove">
  </form>
</body>
</html>
```

```
function removecolor ( )
{
  let x =
    document.getElementById("colorSelect");
    x.remove(x.selectedIndex);
}
```



getElementsByTagName()

- getElementById() allows you to work with elements by their individual id but often you will want to work with a group of elements
- getElementsByTagName() allows you to work with groups of elements. This method returns an array

Exam

```
<html>
```

```
<body>
```

```
<p>Hello World!</p>
```

```
<p>The DOM is very useful.</p>
```

```
<p>This example demonstrates the <b>getElementsByTagName</b> method</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
var x = document.getElementsByTagName("p");
```

```
document.getElementById("demo").innerHTML = 'The first paragraph (index 0) is: '
```

```
+ x[0].innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

Hello World!

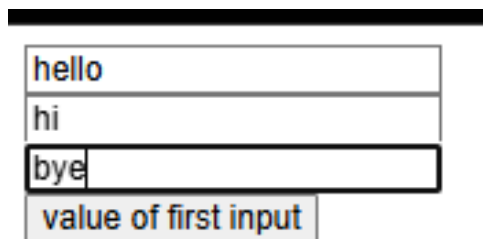
The DOM is very useful.

This example demonstrates the
getElementsByTagName method

The first paragraph (index 0) is: Hello
World!

Document Object: Use getElementsByTagName()

```
<html>
  <head>
    <script type="text/javascript">
      function getElements() {
        var x = document.getElementsByTagName("myInput")[0].value;
        alert("value of first element: " + x);
      }
    </script> </head>
    <body>
      <input name="myInput" type="text" size="20"><br>
      <input name="myInput" type="text" size="20"><br>
      <input name="myInput" type="text" size="20"><br>
      <input type="button" onclick="getElements()" value="value of first input">
    </body>
</html>
```



hello

hi

bye

value of first input

An embedded page at cdpn.io says
value of first element: hello

OK

getElementsByClassName- Example

```
<html>
<body>

<p>Hello World!</p>

<p class="intro">The DOM is very useful.</p>
<p class="intro">This example demonstrates the
    <b>getElementsByClassName</b> method.</p>

<p id="demo"></p>

<script>
var x = document.getElementsByClassName("intro");
document.getElementById("demo").innerHTML =
'The first paragraph (index 0) with class="intro": ' + x[0].innerHTML;
</script>

</body>
</html>
```

Hello World!
The DOM is very useful.
This example demonstrates the
getElementsByClassName method.
The first paragraph (index 0) with class="intro": The
DOM is very useful.

Finding HTML Objects

Property	Description
document.anchors	Returns all <a> elements that have a name attribute
document.forms	Returns all <form> elements
document.images	Returns all elements
document.title	Returns the <title> element
document.URL	Returns the complete URL of the document

Document Object: Return the innerHTML of the first anchor in a document

```
<html>
```

```
  <body>
```

```
    <a name="first">First anchor</a><br />
```

```
    <a name="second">Second anchor</a><br />
```

```
    <a name="third">Third anchor</a><br />
```

```
  <br />
```

InnerHTML of the first anchor in this document:

```
<script type="text/javascript">
```

```
  document.write(document.anchors[0].innerHTML)
```

```
</script>
```

```
</body>
```

```
</html>
```


Document Object: Access an item in a collection

```
<html>
```

```
<body>
```

```
  <form id="Form1" name="Form1">
```

```
    Your name: <input type="text">
```

```
  </form>
```

```
  <form id="Form2" name="Form2">
```

```
    Your car: <input type="text">
```

```
  </form>
```

```
<p> To access an item in a collection you can either use the number or the name of the  
item:</p>
```

```
<script type="text/javascript">
```

```
document.write("<p>The first form's name is: " + document.forms[0].name + "</p>")
```

```
document.write("<p>The first form's name is: " +  
  document.getElementById("Form1").name + "</p>")
```

```
</script> </body>
```

```
</html>
```

Example- anchors

```
<html>
```

```
<body>
```

```
<a name="html">HTML Tutorial</a><br>
```

```
<a name="css">CSS Tutorial</a><br>
```

```
<a name="xml">XML Tutorial</a><br>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML =
```

```
"Number of anchors are: " + document.anchors.length;
```

```
</script>
```

```
</body>
```

```
</html>
```

DOM Manipulation



Attributes

- `getAttribute( attr )` //to get the attribute value
- `setAttribute( attr, value )` //to set the attribute value

Style

- `node.style`

Changing HTML Elements

Method	Description
<code>element.innerHTML=</code>	Change the inner HTML of an element
<code>element.attribute=</code>	Change the attribute of an HTML element
<code>element.setAttribute(attribute,value)</code>	Change the attribute of an HTML element
<code>element.style.property=</code>	Change the style of an HTML element

JavaScript HTML DOM - Changing CSS

`document.getElementById(id).style.property=new style`

```
<html>
  <body>
    <p id="p2">Hello World!</p>

    <script>
      document.getElementById("p2").style.color="blue";
    </script>

    <p>The paragraph above was changed by a
script.</p>
  </body>
</html>
```

Modify paragraph text style through javascript code using button

```
<!DOCTYPE html>
<html>
<head>
<title>JS DOM paragraph style</title>
</head>
<body>
<p id ='text'>JavaScript Exercises - modify
  text style</p>
<div>
<button id="jsstyle"
  onclick="js_style()">Style</button>
</div>
</body>
</html>
```

```
function js_style() {
//font styles added by JS:
  text.style.fontSize = "14pt";
  text.style.fontFamily = "Comic Sans
  MS";
  text.style.color = "green"; }
```

Creating new nodes

name	description
<code>document.createElement("tag")</code>	creates and returns a new empty DOM node representing an element of that type
<code>document.createTextNode("text")</code>	creates and returns a text node containing given text

```
// create a new <h2> node
var newHeading = document.createElement("h2");
newHeading.innerHTML = "This is a heading";
newHeading.style.color = "green";
```

JS

- ❑ merely creating a node does not add it to the page
- ❑ you must add the new node as a child of an existing element on the page...

Modifying the DOM tree

name	description
<u>appendChild</u> (node)	places given node at end of this node's child list
<u>insertBefore</u> (new, old)	places the given new node in this node's child list just before old child
<u>removeChild</u> (node)	removes given node from this node's child list
<u>replaceChild</u> (new, old)	replaces given child with new node

```
var p = document.createElement("p");  
p.innerHTML = "A paragraph!";  
document.getElementById("main").appendChild(p);
```

JS


```
</> HTML
```

```
<ul id="list">  
  <li>Apple</li>  
  <li>Banana</li>  
</ul>
```

```
let newItem = document.createElement("li");  
newItem.innerText = "Orange";
```

Apple

Banana

Orange ← added at end

```
document.getElementById("list").appendChild(newItem);
```

```
let newItem = document.createElement("li");  
newItem.innerText = "Mango";
```

Apple

Mango ← inserted before Banana

Banana

Orange

```
let list = document.getElementById("list");  
let secondItem = list.children[1];
```

```
list.insertBefore(newItem, secondItem);
```

```
let list = document.getElementById("list");  
let firstItem = list.children[0];
```

Mango

Banana

Orange

```
list.removeChild(firstItem);
```

```
</> HTML
```

```
<ul id="list">  
  <li>Apple</li>  
  <li>Banana</li>  
</ul>
```

```
let list = document.getElementById("list");  
  
let newItem = document.createElement("li");  
newItem.innerText = "Grapes";  
  
let oldItem = list.children[1];  
  
list.replaceChild(newItem, oldItem);
```

Mango

Grapes ← replaced Banana
Orange

```
let items = document.getElementsByTagName("li");  
let secondItem = items[1];
```

```
let items = document.querySelectorAll("li");  
let secondItem = items[1];
```

```
let secondItem = document.querySelector("li:nth-child(2)");
```

```
let list = document.getElementById("list");  
let secondItem = list.children[1];
```

```
let list = document.getElementById("list");  
let secondItem = list.querySelector("li:nth-child(2)");
```

- Use `querySelector` / `querySelectorAll` in most cases

DOM Manipulation

Insert Elements

let el = document.createElement("div")

- node.append(el) //adds at the end of node (inside)
- node.prepend(el) //adds at the start of node (inside)
- node.before(el) //adds before the node (outside)
- node.after(el) //adds after the node (outside)

Delete Element

- node.remove() //removes the node

appendChild() adds only a single node at the end of a parent, whereas **append()** can add multiple nodes or text and is more flexible.

- `document.body.append("Hello", document.createElement("p"));`



What will it create?



It will add TWO things inside `<body>`:

1. "Hello" → Text Node
2. `<p>` → Empty Element Node

```
<body>
  Hello
  <p></p>
</body>
```

`document.body.appendChild("Hello");` **// ERROR**

- `appendChild` only accepts nodes
- "Hello" is just a string

DOM Manipulations

● **append() / appendChild()**

(Add INSIDE at the end)

```
<div id="box">
  A
  B ← added here
</div>
```

● **prepend()**

(Add INSIDE at the start)

```
<div id="box">
  B ← added here
  A
</div>
```

● **before()**

(Add OUTSIDE before the element)

```
B ← added here

<div id="box">
  A
</div>
```

● **after()**

(Add OUTSIDE after the element)

```
<div id="box">
  A
</div>

B ← added here
```

Practice Question

. Create a new button element. Give it a text “click me”, background color of red & text color of white. Insert the button as the first element inside the body tag.

Practice Question

. Create a new button element. Give it a text “click me”, background color of red & text color of white. Insert the button as the first element inside the body tag.

```
let newBtn = document.createElement("button");  
newBtn.innerText = "click me!";  
  
newBtn.style.color = "white";  
newBtn.style.backgroundColor = "red";  
  
document.querySelector("body").prepend(newBtn);
```


Practice Question

- Create a <p> tag in html, give it a class & some styling.

Now create a new class in CSS and try to append this class to the <p> element.

Did you notice, how you overwrite the class name when you add a new one?

Solve this problem using classList.

```
<body>
  <p class="content">i am a paragraph.</p>
  <script src="script.js"></script>
</body>
```

```
.content {
  color: red;
}

.newClass {
  background-color: green;
}
```

i am a paragraph.

i am a paragraph.

Practice Question

- Create a <p> tag in html, give it a class & some styling.

Now create a new class in CSS and try to append this class to the <p> element.

Did you notice, how you overwrite the class name when you add a new one?

Solve this problem using classList.

```
<body>
  <p class="content">i am a paragraph.</p>
  <script src="script.js"></script>
</body>
```

```
.content {
  color: red;
}

.newClass {
  background-color: green;
}
```

i am a paragraph.

i am a paragraph.

```
< <p class="content">i am a paragraph.</p>
> para.getAttribute("class");
< 'content'
> para.setAttribute("class", "newClass");
```

Practice Question

- Create a <p> tag in html, give it a class & some styling.

Now create a new class in CSS and try to append this class to the <p> element.

Did you notice, how you overwrite the class name when you add a new one?

Solve this problem using classList.

```
<body>
  <p class="content">i am a paragraph.</p>
  <script src="script.js"></script>
</body>
```

```
.content {
  color: red;
}

.newClass {
  background-color: green;
}
```

i am a paragraph.

i am a paragraph.

```
< <p class="content">i am a paragraph.</p>
> para.classList.add("newClass");
```